

- (1) Password security strength of a password in bits? 取log2 比如 $\log_2 2^{62^{10}} \approx 60$ bits suggested key strength for symmetric key encryption is at least 80 bits (128 bits recommended).
- 字典、暴力、彩虹表攻击的区别:
- Dictionary: Pre-compute hash of all possible password combinations (also could be called a look-up table, sometime refers to list of only known words/common passwords)
 - Brute force: Try to search for all entries in file (cover all password combinations)
 - Rainbow table: More efficient than Dictionary in terms of storage. You store hash chains (start and end point). If you get a hash X, keep rehashing it until you match an endpoint. Recreate the chain from the start point. The password in the chain entry preceding X. 使用预先计算的哈希值和明文密码的映射表, 直接匹配目标密码的哈希值
- (2) Phishing 预防技术手段 (除了 educated user)
- Phishing mostly target passwords, which is the "something user knows" authentication factor. We can therefore protect a user more if we use a 2nd factor, like "something a user has".
- At the same time we could use one-time login information in combination with something user has, e.g. sending a one time use PIN to user mobile, having OTP generators, etc..
- 引入第二个因素或者将一次性登录信息与用户拥有的物品结合使用。例如向用户手机发送一次性 PIN 码

Lecture 9 Network Security

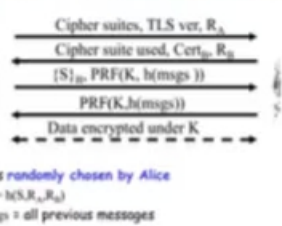
1. Secure Socket Layer (SSL)

SSL is the protocol used for most secure transactions over the Internet.

One-way authentication, data confidentiality, authentication need not be mutual

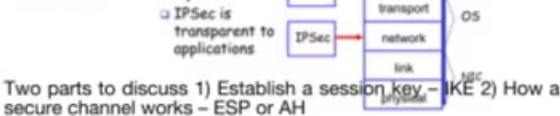
- SSL designed for use with HTTP 1.0
- HTTP 1.0 usually opens multiple simultaneous (parallel) connections
- SSL session establishment is costly

Simplified SSL Handshake Protocol



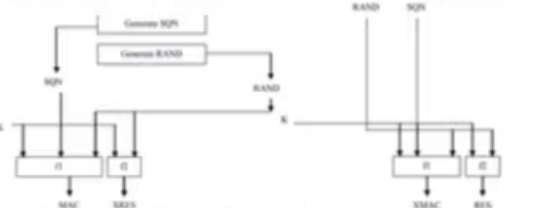
'msgs' is the previous message exchanged between Alice and Bob in the handshake. Authentication: Bob to Alice. Key control: Both. Key confirmation: Yes (PRF is pseudo-random functions (like a hash).)

2. IPsec



Two parts to discuss 1) Establish a session key - IKE 2) How a secure channel works - ESP or AH

- (4) WEP 碰撞: IV 的长度是 24 位, 因此理论上 $2^{24} = 16,777,216$ 种可能的取值, 开根号约等于 4040, 这意味着大约发送 4,040 个数据包后, IV 碰撞的概率接近 50%.
- WEP 的已知明文攻击就是 RC4 的方式: C=M 得到密钥流使用 WEP 的公共 WIF 的防护措施: 用 VPN with IPsec, 确保 TLS/SSL (https) connection
- (5) Mobile network
- 2 代的服务: ①Authentication of mobile station (phone)
- ②Encryption (only between phone and base station) (使用专有密钥 A3/A5/A8)
- 3 代: ①Mutual authentication ②Encryption (plus integrity check MAC) Extends to base station controller (BSC) AES



- AuC (Authentication Center) 生成
- ①生成随机数 (RAND): 用于挑战移动设备, 确保认证过程的随机性, 防止重放攻击。
 - ②生成序列号 (SQN): 用于同步和检测重放攻击。
 - ③共享密钥 K: 事先在 AuC 和移动站中预共享的密钥。
- ④计算 f1 和 f2:
- f1: 使用 K 和 SQN 计算认证码 MAC (Message Authentication Code) $MAC = f1(K, SQN, RAND)$
- f2: 使用 K 和 RAND 计算认证响应 XRES (Expected Response) $XRES = f2(K, RAND)$
- 发送挑战: AuC 将 RAND、SQN 和 MAC 发送给移动站
- 移动站 (Mobile Station) 验证
- ①接收 RAND 和 SQN: 从 AuC 发送来的挑战随机数和序列号
 - ②共享密钥 K: 与 AuC 预共享的密钥。
 - ③计算 f1 和 f2:
- f1: 计算本地认证码 XMAC, $XMAC = f1(K, SQN, RAND)$
 - f2: 计算本地响应 RES 回应 AuC 认证挑战, $RES = f2(K, RAND)$
- 验证 XMAC 是否等于收到的 MAC, 如果相等, AuC 的身份被认证。
- 移动站回应: 将 RES 发送回 AuC
- AuC 验证: 验证接收到的 RES 是否等于 XRES, 如果相等, 移动站的身份被认证。

Lecture 10 Security Threats and Mechanisms

Services	Threats	Mechanisms
Confidentiality 保密性	data disclosure	encryption
Integrity 完整性	data alteration	MAC/digital signature
Availability 可用性	Dos	Redundancy
Entity Authentication	masquerade	authentication protocol
Origin Authentication	forgery	MAC/digital signature
Non-repudiation 不可否认	repudiation	digital signature
Access control	illegitimate access	Access control model

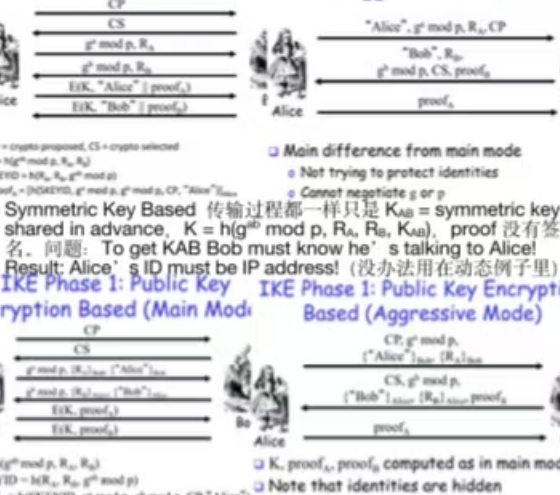
Lecture 2 对称密码

1. 各种密码
- 凯撒密码: Caesar Cipher, 移位密码
 - 替换密码: 如 $H \rightarrow Q, E \rightarrow W, L \rightarrow X$, e 和 t 频率最高
 - Vigenere Cipher: 多表替换密码, 相同的明文可以加密成不同的。例如: 选择一个单词 pauli 则 $a_1 \rightarrow p, a_2 \rightarrow a, a_3 \rightarrow u$
 - One-time Pad Encryption: Plaintext $\oplus$ Key = Ciphertext
- Good: The ciphertext can decrypt to any possible plaintext

IPsec 的原理是通过协议 (如 AH 和 ESP) 对 IP 数据包进行加密和认证, 确保数据的机密性、完整性和来源可信, 同时通过密钥交换协议 (如 IKE) 动态协商安全参数。

3. IKE

- Phase 1 — master session key setup.
- 3 ways: Public key encryption based, Signature based, Symmetric key based.
- 2 modes: Main mode, Aggressive mode
- IKE Phase 1: Signature Based (Main Mode)
- IKE Phase 1: Signature Based (Aggressive Mode)



问题: 第三者可以自导自演假装 Alice 和 Bob 伪造一次会话

4. ESP AND AH

Transport Mode: 原样发送原始 IP 报头, 可以看到谁是真正的发送者和接收者。(仅加密数据部分, host to host)

Tunnel Mode: 加密整个 IP 数据包, 包括原始 IP 头 (原始 IP 报头被视为数据, 对 attacker 不可见, 不知道收发者), 新增 IP 头 (Router-to-router / Gateway-to-gateway, 相当于隐藏源 src ip 和 dest ip, 并把他们改成网关的 ip)

Authentication Header (AH): ①Provides message authentication. ②Next header: TCP, UDP, etc 完整性校验 + 认证来源, 不加密 Encapsulating Security Payload (ESP): ①Provides confidentiality and authentication. Either is optional. ②Either encryption or authentication (or both) must be enabled 加密数据+完整性校验

5. AH 协议

Transport Mode: 仅对 IP 数据包的有效载荷 (如 TCP 数据) 进行认证, 外加对 IP 头中不可变字段 (immutable fields) 进行认证

Tunnel Mode: 认证范围是 封装后的数据包, 包括原始 IP 数据包 (IP 头 + 数据) 和 新的外部 IP 头的不可变字段。

*争议: Tunnel Mode 下原始 IP 头被新的 IP 头取代, 但原始 IP 头仍保持明文, 且 AH 协议仅提供数据来源认证 (不加密数据), 从表面上看, Tunnel Mode 可能显得“无用”, 因为它并不隐藏原始数据包内容。

- 实际作用: Tunnel Mode 的优势是它能够对整个原始 IP 数据包的完整性进行保护, 包括原始 IP 头和有效载荷。

Bad: Pad must be random, used only once. Pad has the same size as message 逐位异或

- ① Stream Ciphers: 流加密非常快, 但容易受到密钥重用攻击
- (1) Secret key length: 128 bits, 256 bits, etc.
 - (2) Maximum plaintext length: usually can be arbitrarily long
- Key reuse: $C1 \oplus C2 = (M1 \oplus K) \oplus (M2 \oplus K) = M1 \oplus M2$
- 攻击者通过对比 M1 和 M2 的相似结构可以推断出明文的信息
- 缺点: 如果攻击者不知道明文, 但可以修改密文, 可翻转一位修改接收方得到的明文。如果知道明文 P: $KS = C \oplus P$ 可得到密钥流, 则可以伪造密文, 所以需要 integrity 和 data origin authentication
- ② RC4 流密码: 利用一个可变的变状态数组和密钥流生成一组伪随机字节, 用来与明文异或, 生成密文。(前 256 字节建议丢弃)
2. 分组密码
- 概述: 将明文块划分为固定大小的块, 并对每个 block 独立加密
- DES: 64 位明文块, 16 轮加密, 有效密钥为 56 位 (总长 64 位), 每个轮密钥 Ki 都参与对应轮次的加密, 最终置换生成 64 位的密文。暴力搜索空间 2^{56} , 暴力破解平均搜索次数 2^{55} , 即 2^{28}
 - 3DES: K1 加密, K2 解密, K1 再加密。两种模式: ①两密钥 (K1, K2, K1, 112 位) ②三密钥 (K1, K2, K3, 168 位)
 - DESX: 三密钥 $C = K3 \oplus DES(K2, M \oplus K1)$, K1, K3 长度和明文相同, K2 是标准的 DES 的 56 位密钥
 - AES: 块大小 128 位, 密钥长度为 16, 24, 32 字节 (128, 192, 256 位) 依照密钥长度有 10~14 轮加密。

3. Block Cipher 的工作模式

- MSB 是最高位, LSB 是最低位
- ECB: 每个块独立加密 $C = E(K, P)$ $P = D(K, C)$ 会受 cut and paste 攻击: 可移动, 删除密文块
 - CBC: IV 公开但随机, 相同的明文不会产生相同的密文。错误: ①P0 变了, 所有块都会发生变化。②1 位密文错误会导致: 当前密文块解密完全错误, 下一密文块解密中有 1 位错误 ③接收方 IV 不正确, 只有解密的第一个明文块会受到影响。
 - $C0 = E(K, IV \oplus P0)$ $P0 = IV \oplus D(K, C0)$ $C1 = E(K, C0 \oplus P1)$ $P1 = C0 \oplus D(K, C1)$
 - CTR: CTR 模式允许并行加密和解密, 适合高性能应用。错误: 1 位密文错误只影响当前块 1 个块丢失会影响后面所有
 - $C0 = E(K, IV) \oplus P0$ $P0 = C0 \oplus E(K, IV)$ $C1 = E(K, IV \oplus P1)$ $P1 = C1 \oplus E(K, IV \oplus 1)$
 - CFB: 可以将分组密码变为流密码, 适合对逐字节数据流的加密 $C0 = E(K, IV) \oplus P0$ $P0 = C0 \oplus E(K, IV)$ $C1 = E(K, C0) \oplus P1$ $P1 = C1 \oplus E(K, C0)$
- 错误: 和 CBC 类似, 1 位错误影响两个块, IV 错误影响第一个块

Lecture 3 Number Theory (好像不考)

1. 扩展欧几里得算法
- 求 $ax + by = \gcd(a, b)$ 的整数解, 求模逆
- 例: $a = 911, b = 999$
- $999 = 911 \cdot 1 + 88$ $911 = 88 \cdot 10 + 31$ $88 = 31 \cdot 2 + 26$ $31 = 26 \cdot 1 + 5$ $26 = 5 \cdot 5 + 1$ 由最后的余数 1 得出 $\gcd(a, b) = 1$
- 求模逆: $1 = 26 - 5 \cdot 5 = 26 - 5 \cdot (31 - 2 \cdot 6) = 26 - 5 \cdot 31 + 6 \cdot (88 - 31 \cdot 2) = 88 \cdot 6 - 31 \cdot 17 = 88 \cdot 6 - (911 - 88 \cdot 10) \cdot 17 = 88 \cdot 178 - 911 \cdot 17$
- 得到 $-193, 911 \bmod 999$ 的逆元 $= -193 + 999 = 806$
2. 欧拉函数
- 若 n 是质数 $\phi(n) = n - 1$ 合数: $\phi(mn) = \phi(m) \cdot \phi(n)$
3. 费马小定理
- 若 p 为素数且 a 不是 p 的倍数则 $ap - 1 \equiv 1 \bmod p$
- 欧拉推广, 若 n 是合数, 对于任何与 n 互质的 a, $a\phi(n) \equiv 1 \bmod n$
4. 模幂
- 在模 n 的条件下计算 $a^x \bmod n$, 两种方法: 重复乘法、平方乘法
- Lecture 4 PKE
- PKE 基于数学的单向函数: 给定输入, 可以输出容易, 从输出到输入很难, 因数分解问题 (Factorization 给两个质数的积, 难以

6. ESP 协议

Transport Mode: (原始 IP 头明文存在)

加密范围: TCP/UDP 头部 + 数据部分。

认证范围: 从 ESP 头开始 (原始 IP 头大部分字段不受保护)。

Tunnel Mode: (加密整个原始 IP 数据包, 保护原始 IP 头)

加密范围: 整个原始 IP 数据包 (包括原始 IP 头和数据)。

认证范围: ESP 头 + 加密部分。

新 IP 头的可变字段 (如 TTL) 不受认证保护。

7. GSM security

GSM (2G) 安全性 提供基本的身份验证、机密性和匿名性服务。

3G (UMTS) 安全性 在 GSM 基础上引入了双向认证, 用户和网络相互验证身份, 避免伪基站攻击。

8. WLAN Security

- WEP (Wired Equivalent Privacy):
- 使用长期密钥和 RC4 加密算法, 24 位初始化向量 (IV)。
 - 存在密钥重用和弱的完整性校验机制 (CRC32), 极易被破解, 非常不安全。
- WPA (Wi-Fi Protected Access):
- 改进了密钥管理, 通过临时密钥 (TKIP) 解决密钥重用问题。
 - 仍然使用 RC4 加密算法, 但增加了动态密钥生成, 提高安全性。
- WPA2:
- 使用 AES 加密算法 取代 RC4, 提供更强的数据加密。
 - 采用 CCMP (基于 AES 的加密和完整性保护) 机制, 增强数据完整性, 成为现代 WLAN 安全标准

9. DOS 和 DDOS

DoS (拒绝服务) 攻击 是通过单一攻击源发送大量恶意请求, 耗尽目标资源, 使其无法响应正常用户。

DDoS (分布式拒绝服务) 攻击 则利用多个分布式攻击源 (如僵尸网络) 同时发动更大规模的攻击, 导致目标系统更难抵御和追踪。

Key concept for DoS attacker is resource amplification

防御: 部署防火墙和入侵检测系统 (IDS)、实时监控网络流量, 设置阈值警报, 启用速率限制 (Rate Limiting) 以防止流量激增, 使用内容分发网络 (CDN) 分散流量, 部署防御服务 (如 Cloudflare, Akamai) (后 2DDos 高级措施)

10. Tutorial 10

- (1) Firewall rules
- 最后一条是 default rule 拒绝
- (2) What would TLS look like if it was based on symmetric cryptography only? Would it be practical?
- No. $\{S\}_K$ and key based on long term symmetric key K_{AB} . $K = h(K_{AB}, R_A, R_B)$ This would require every client to somehow share a symmetric key with every server on the Internet... not practical
- The purpose of the message $E(K, h(msgs \| K))$ sent?
- 确认 A 知道 K (显式密钥认证) 验证消息完整性并防止重放攻击, 不能删除 $E(K, h(msgs \| K))$ 的原因: 会话密钥 K 无法传达给 Bob 容易被 Dos 攻击, 攻击者可以伪造消息发送 $\{S\}_K$ 给 Bob, 然后立即断开连接, 导致 Bob 保持一个等待状态。
- (2) cookie 在 IKE 的作用: These cookies can help identifying spoofed packets (i.e., packets containing a fake sender address). This ties the messages to a specific IP address of A and B. 需要在计算前确认 cookie 有效不然会引起昂贵运算, Dos 攻击。
- (3) padding 的作用: ① If the encryption algorithm used is a block cipher and not a stream cipher then the padding is used to expand the plaintext to a multiple of the block size. ②Padding may also be used to conceal the actual length of the payload by adding a certain amount of padding

找回这两个质数) 离散对数问题 (DLP 给 $z = a^b \bmod n$, 找 b 很难)

1. RSA 加密

- 准备: ①选择两个大素数 p 和 q 计算 $n = p \cdot q$ ②计算欧拉函数 $\phi(n) = (p-1)(q-1)$ ③选择公钥指数 e, 满足 $1 < e < \phi(n)$ 且 $\gcd(e, \phi(n)) = 1$ ④计算私钥指数 d, 使得 $e \cdot d \equiv 1 \bmod \phi(n)$ ⑤公钥: (e, n) 私钥: d (私钥是模逆计算得到)
- 加密: ①将消息 M 转换为整数, 且 $M < n$ ②计算密文 $C = M^e \bmod n$
- 解密: 计算明文 $M = C^d \bmod n$ (依赖因数分解问题)
2. ElGamal 加密
- 依赖离散对数问题: 选一个大素数 p 和生成元 g
- 私钥 x 是随机选择的整数, 公钥 $y = g^x \bmod p$
- 加密: 随机选择 r, 生成密文 $C = (A, B) = (g^r \bmod p, M \cdot y^r \bmod p)$
- 解密: 计算 ① $K = A^x \bmod p$ ② $M = B \cdot K^{-1} \bmod p$ (K 的逆元)

3. Diffie-Hellman 密钥交换

- ①Alice 给 Bob 发送 $g^a \bmod p$, Bob 给 Alice 发送 $g^b \bmod p$
- ②A 计算 $(g^b)^a \bmod p$, B 计算 $(g^a)^b \bmod p$ ③ $K = g^{ab} \bmod p$
- 容易受到中间人攻击
- A 发 $g^a \bmod p$ Trudy 接收, Trudy 给 Alice 发送 $g^t \bmod p$
- Trudy 发送 $g^t \bmod p$ Bob 接收, Bob 给 Trudy 发送 $g^b \bmod p$
- Trudy 和 A 共享密钥 $g^{at} \bmod p$, Trudy 和 B 共享 $g^{bt} \bmod p$

Lecture 5 数字签名与身份验证

1. RSA 签名

- 准备与加密一样: ①签名生成: $S = M^d \bmod n$, M 是消息
- ②签名验证: 如果 $S^e \bmod n = M$ 说明验证成功

2. Hash Function

- ①如果 RSA 签名方案 $M > n$, 不直接签名 M, 而是对 $h(M)$ 进行签名
- Alice sends M and $S = \text{SignSK}_{\text{Alice}}(h(M))$ to Bob
- Bob verifies that $\text{VerifyPK}_{\text{Alice}}(h(M), S) = \text{valid}$
- 哈希函数安全性计算: 根据生日悖论, 对于输出长度为 (N) 位的哈希函数, 找到碰撞的复杂度约为 $(2^{N/2})$ 次运算。例如, SHA-256 的碰撞复杂度约为 (2^{128})
- ②基于分组密码的 Hash 构造:
- ①初始化 Hash 值: 定义初始向量 IV, 为固定值, 大小等于块
 - ②分块消息和块大小一致, 最后用 0 填充
 - ③逐块处理消息, 对每个块 M_i , $H_{i+1} = E(K, H_i \parallel M_i)$, $E(K, x)$ 是块加密函数, H_i 是第 i 步的 hash 值, H_0 为 IV
 - ④输出哈希值: H 为整个消息的 hash 值

3. MAC (Message Authentication Code 消息认证)

接收方使用消息和 K 计算 MAC 标记; 然后将其与收到的 MAC 标记进行比较, 如果它们相等, 则接收方得出结论, 消息未更改

假设有一个明文序列 $P = P_0, P_1, P_2, \dots$ 可使用共享密钥 K 计算:

$C_0 = E(K, P_0)$ $C_1 = E(K, C_0 \oplus P_1)$ $C_2 = E(K, C_1 \oplus P_2)$, 最终 MAC tag 是 $C_n = E(K, C_{n-2} \oplus P_{n-1})$ (CBC-MAC 修改任何一个块都改变 tag)

分组密码的不安全性: $IV = 0$, 给定两对消息和标签 $(P', T'), (P'', T'')$, 其中 $P' = P_1, P_2, P'' = P_3$, 在不知道密钥的情况下, 攻击者可以设置 $P''' = P_1, P_2, P_3 \oplus T'$ 和 $T''' = T'$ 生成一对新的 (P''', T''') 与 hash 相比: 两者都将任意长度的消息映射到固定长度的输出与签名相比: 更快, 但 MAC 不提供不可否认性

4. HMAC (用 Hash 函数实现的 MAC)

Hash($\text{opad} \parallel \text{key} \parallel \text{Hash}(\text{ipad} \parallel \text{key} \parallel \text{message})$), $\parallel$ 为连接符号

$\text{ipad} = 00110110(0x36)$ $\text{opad} = 01011100(0x5c)$ 不断循环到分组长若密钥长度 < Hash 分组长度 key = 密钥 000... (末尾用 0 填充), 使用步骤: 填充 0, 计算内部 Hash, 计算外部 Hash

Padding 的作用: ① 使消息长度符合哈希函数处理的块大小要求。②通过添加消息长度信息, 确保不同长度的消息产生不同哈希值, 减少冲突。

Lecture 6 Authentication

1. Challenge-Response One-Way Authentication



使用 Public Key Based: B 发送 $(R)_{Alice}$, A 用自己的私钥解密 R 发送给 B; 或者 B 发送 R, A 发送用自己私钥加密的 $(R)_{Alice}$

2. Entity authentication

实体身份验证的要求: 确信消息来自主体, 确信消息是新的
- Claimant 申请人: An entity claiming an identity.
- Principal: The identity claimed by a claimant.
- Verifier 验证者: The entity verifying a claim.
The claimant is the person the verifier is talking to, the principal is the logical identity the claimant says he is.
• Unilateral authentication: 单方面实体认证
• Mutual authentication: 互相认证

3. Authentication attacks

Masquerade: 伪装攻击是攻击者直接生成表明他们是其他人的消息的攻击。(origin authentication)
Replay: 重放攻击是将旧消息重放给验证者的攻击。(freshness)
Reflection: 反射攻击是一种将验证者生成的数据发送回给他的攻击。(在第一个会话得到 R, 在第二个会话发送 R 让验证者发回 MAC, 得到消息认证码 MAC 再发回第一个会话)
- Prevented by including identifiers that show to whom a message is being sent. (在消息中包含一个标识符, 说明谁被谁识别。如果 MAC 是根据 nonce 和被验证的人的名字计算的, 那么上述攻击就不会起作用)

4. Protocol design/analysis

需要区分 security objectives 和 protocol goals, 前者是 real world problem, 后者是 technical requirement.

Very simple example

Defining the security objectives

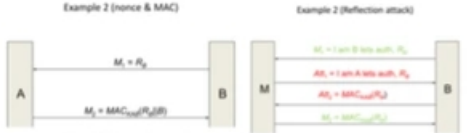
- Bob wants to make sure that Alice was the source of a electronic purchase contract.
 - Bob wants to the contract to be enforceable at later date (Alice should not deny it)
- Determining the protocol goals
- Bob requires data origin authentication of the message received from Alice
 - Bob requires non-repudiation of the message received from Alice

例子: 用 Timestamp



B checks three things:
• the deciphered message 'makes sense' (has the appropriate redundancy) – comes from possible MDC, is sent to B and good guess for TA, – the time-stamp is within its current window (and,

using its 'log', that a similar message has not recently been received), • B's name is correctly included.



Assumptions? K_{AB} shared Goals? Mutual or unilateral? Freshness come from? Nonce B Data origin authentication? MAC It is based on the use of nonces (for freshness) and a data integrity mechanism (for origin and integrity checking). It provides unilateral authentication (B can check A's identity, but not vice versa). When B sends the message M_1 , B stores the nonce R_{AB} . When B receives message M_2 , B first assembles the string $R_{AB}|B$ and then computes $MAC_{K_{AB}}(R_{AB}|B)$, using the shared secret K_{AB} . B checks one thing: the newly computed check value agrees with the one in message M_2 . (如果没有 B 可能会被 Reflection attack. 如果 MAC 改成用 A 的私钥进行 Sig 签名的话也可以防止该攻击)



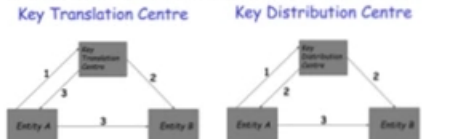
***交换非重复数 (Nonce) $(R_A|R_B)$ 变成 $R_B|R_A$ 可防止 replay: A 发送 R_A , M 发现这个 R_A 之前 B 用过, 等于 R_B , M 又有之前的 M_2 , 所以 M 把 R_B 设置为之前的 R_A , 则发送给 A 的 $M_3 = R_B|Sig_A(R_B|R_A)|B = R_A|Sig_B(R_A|R_B)|B = M_2$, 因此攻击成功。

5. Tutorial 6

- (1) 考虑用 Nonce 还是 Timestamp, 如果不能保证时间正确 (not guaranteed to have right time) 就不能用 Timestamp!
- (2) Nonce 可以是 random 或者 counter, 但 counter 可能被预测, 一般只在安全、受控环境使用。
- (3) 银行提供的安全设备, 按下后计算是自动的: 比如设备内部预先存储了一个共享密钥 KDB, 仅服务器 (B) 和设备 (A) 知晓。按钮按下后, 设备自动执行以下操作: 加密或生成 MAC: 使用共享密钥 K_{DB} 对收到的挑战信息 R_B 进行加密或生成消息认证码 (MAC)。

Lecture 7 Key Management

- 1. Symmetric-key protocols
 - o Directly communicating entities (两个实体通过安全信道直接通信以建立密钥。例如, 使用现有的共享密钥或相互信任的公钥副本)
 - o Use of a Key Distribution Centre (KDC) 可信, 可生成密钥并分发
 - o Use of a Key Translation Centre (KTC) 可信, 可 transport 密钥



(5) Payment Card Example

- Use public key crypto – use certificates!
o Step 1: Card signs transaction data and also sends the card's certificate • Card certificate signed by card vendor
o Step 2: Terminal has card company certificate, so verifies card certificate, then verifies card signature
- Use public key cryptography and certificates. POS >> Card: R_{AB} . Verify transaction data M Card >> POS: $Cert_{Card}(R_{AB}, M)_{Agent, Card}$
- Merchant POS stores certificates for issuers (banks or credit card company)
- Card stores its private key, and public key cert – Cert is signed by its issuer
- Card also stores symmetric key (K)
– Key is shared with issuer
– Key is unique to card

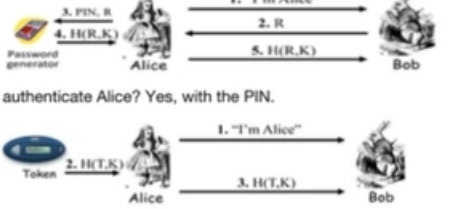
Lecture 8 Computer Security

- 1. Access Control Authentication+authorisation = Access Control authentication (who are you) and authorisation (what are you allowed to do).

- 2. Password File Given password, choose random s, compute $y = h(\text{password}, s)$ and store the pair (s,y) in the password file
□ Note: The salt s is not secret, 这样可以防止字典攻击 (Attacker must recompute dictionary hashes for each user 计算量变大很多)

- 3. Phishing Attacker masquerades as a trustworthy entity
o Attempts to fraudulently acquire sensitive information (usernames, passwords and credit card details)
o Convince user to perform some other actions important aspects of successfully executing this attack is. Failure of authentication in itself...

- 4. 2-factor Authentication Requires 2 out of 3 of Something you know, Something you have, Something you are
□ Examples: o Password + Security Token o ATM: Card and PIN o Password + Cellphone (e.g. SMS)



Time synchronization between the token and Bob is required. authentic Alice? No, only that 'Alice' has control of the token.

5. Firewall

Firewall must determine what to let in to internal network and/or what to let out □ Access control for the network Types of firewalls

KTC 从 A 接收密钥, 该密钥已经使用 A 和 KTC 共享的密钥加密。KTC 将其解密, 并使用与 B 共享的密钥重新加密。A 请求 KDC 生成并分发给 A 和 B 共享的密钥。情况 1: KDC 将新生成的密钥发送给 A 和 B, 新密钥使用 A 和 B 与 KDC 共享的密钥进行加密。情况 2: KDC 使用 A 和 B 与 KDC 共享的密钥加密新生成的密钥, 它加密的两个加密版本发送给 A, A 恢复其自己的加密版本, 并将根据 B 与 KDC 共享的密钥加密的版本发给 B。

2. 密钥管理术语

- Key establishment: Process of making a secret key available to multiple entities.
- Key agreement: Process of establishing a key in such a way that neither entity has key control.
- Key transport: Process of securely transferring a key from one entity to another.
- Key control: Ability to a choose a key's value.
- Key confirmation: Assurance that another entity has a particular key.

Implicit/Explicit key authentication: 区别在是否抗中间人攻击, Diffie-Hellman 和基于证书的 Diffie-Hellman 密钥交换的区别, 隐式认证意味着协议能够确保密钥 K 和通信双方的身份绑定, 但没有通过后续步骤明确确认密钥的接收

3. 引入可信第三方的例子

“密钥分发中心”模型的示例, 并且其中 A 将密钥“转发”给 B, 注意, 在这个协议中, T 是 A 和 B 都信任的第三方 (KDC), 此外, T 分别与 A 和 B 共享秘密密钥 KAT 和 KBT, Freshness: Nonces

Data origin authentication: Encryption, redundancy TTP is authenticated to A and B. A, B mutually authentication Why are the nonces swapped around in M5? Else you just send back what was sent to B in M4.

Keying material: F

What key establishment security? Key confirmation So is this implicit or explicit key authentication? – B confirms use of key, so does A. Explicitly, Who has key control?

4. Public Key Certificate Management

CA 的 4 个作用: ① Identifying entities before certificate generation. ② Generating user key or verifying user key. ③ Ensuring the quality of its own key pair. ④ Keeping its private key secret.

5. PGP – Anarchy Model

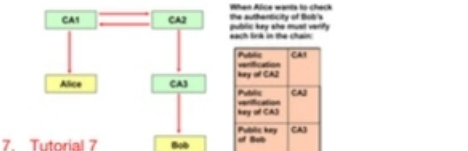
Unstructured

- o Suppose a public key is received and claimed to be Alice's.
- o The public key and Alice's identity are signed by some others (CAs). Each signature is considered as a certificate:
 $Cert_{T_{Alice}}(Alice), Cert_{T_{Alice}}(Alice), Cert_{T_{Alice}}(Alice)$
- Example: If my trust in certificates issued by Bob, Carol, and Dave (whose public keys I already have valid copies) are 1/2, 1/3, 1/3, respectively (and I don't have Eve's public key), then the above public key for Alice is considered as trust-worthy as $1/2 + 1/3 + 1/3 \geq 1$
- o Scalability weakness:
 - Trust is not transferable. Alice trusts Bob, Bob trusts Carol, does not necessarily mean that Alice trusts Carol.
 - It may be cumbersome for one to get adequate certificates so that one's public key can be trusted.

6. PKI Trust Models

(1) Alice gets signed message from Bob and his cert. (2) She verifies the signature with key on Bob's cert (3) She verifies Bob's cert with key on CA3 cert (4) She verifies CA3's cert with key on CA2 cert (5) She verifies CA2's cert with key on CA1 cert (CA1 and CA2 cross certified) (6) She trusts CA1, therefore she trusts all the way down to Bob. This is a certificate chain.

Certificate Chains



7. Tutorial 7

- (1) 是否支持会话密钥 (Session Key) 建立, 看协议是否明确传递或协商了密钥。如 $Enc(A|K)$, K 是动态生成的, 则 K 可能是。
- (2) Long-Term Secret: 私钥或预共享密钥
- (3) Key hierarchy: Top level, bottom level 整个系统的有效安全性取决于整个密钥层次中最弱的部分, 因为攻击者可以通过突破最弱的环节来破坏整个系统。例如 If K_{AT} and K_{BT} are 56-bit DES keys and K is a 256-bit AES key what is the effective security of key K? It can only be as secure as the key that is being used to distribute it so 2^{56}



8. Tutorial 8

- (1) Use asymmetric cryptographic to do key transport between only two parties. – Use public key encryption for one party to send key to the other. Use asymmetric cryptographic to do key agreement between only two parties. – Use Diffie-Hellman, both parties contribute towards the key value (2) A CA's certificate is signed by another CA (cross certification) or by a more powerful CA (the root CA). Alternatively the CA itself signs the key. These are so-called self-signed certificates, because they use their own private key to sign their certificates.

(3) Web PKI

- 浏览器连接到服务器: 浏览器向服务器发出请求, 服务器返回其证书。
- 证书验证: 浏览器使用内置的 CA 公钥验证服务器提供的证书签名, 确保证书确实由受信任的 CA 签发。
- 安全连接建立: 如果验证成功, 浏览器信任服务器的公钥 PK 并使用该公钥协商一个对称密钥 (例如用于 AES 加密)。后续的通信将通过这个对称密钥进行加密, 确保数据机密性和完整性。
- (4) 恶意 CA: 恶意 CA 可以为任何网站签发合法看似合法的证书。浏览器会因为信任该 CA 而认为证书有效, 导致用户无法区分真正的安全网站和恶意网站。攻击者解密用户与其建立的 HTTPS 连接, 同时与真实服务器建立独立的加密连接, 攻击者解密用户发来的 HTTPS 数据 (例如用户名、密码、电子邮件内容等), 这些数据以明文形式对攻击者

- ① self address to visit <https://mail.google.com> and types the address into the browser.
- ② The malicious CA intercepts the attempt, and realises that John tries to access Google Mail. It then (on-the-fly) generates a new key and a corresponding certificate, signs it with its own key, and sends the certificate to John.
- ③ John's browser receives the certificate and checks the issuer. Because the issuer of the certificate (i.e., the malicious CA) is in the list of trusted CAs, the browser will typically accept the certificate without any other formality whatever button will make the notification go away?
- ④ John is now browsing Google Mail, but instead of his connection being secure, it is in fact compromised and the malicious CA can read everything.
- Modern web browser may have some limited protection against this, for example by alerting the user if a certificate for a website suddenly changes. But ask yourself, if you get a notification about some problem with a certificate when connecting to a website, do you actually read the text carefully, or do you just click whatever button will make the notification go away?